USER MANUAL

DSSE Workbench

Distribution System State Estimation workbench: network modelling, power flow, weighted least squares state estimation, bad data analysis, measurement chain simulation and multi-agent estimation, in a single desktop application.

Audience Engineers and researchers using the application; no prior familiarity with the source code is assumed.
Screenshots captured 2026-09-09 11:54
Institution Université Grenoble Alpes · CNRS · G2Elab

Contents

GETTING STARTED

1. About this manual
2. What the application does
3. Installation and startup
4. The application window

WORKING WITH THE APPLICATION

5. Defining the network
6. Running a power flow
7. Estimating the state
8. Bad data analysis
9. Running several methods at once
10. The measurement chain
11. The operator view
12. Multi-agent estimation
13. Built-in reference

REFERENCE

14. Conventions and shortcuts
15. Troubleshooting
16. Maintaining this manual

1 About this manual

This manual describes how to operate the DSSE Workbench. It is organised in the order in which the application is normally used: the network is defined first, a power flow establishes the reference state, the estimator is run on measurements derived from it, and the remaining chapters cover bad data analysis, the measurement chain, the operator view and the multi-agent framework.

Each chapter states the purpose of the corresponding tab, shows the relevant part of the interface, and lists the controls in the order in which they are used. The numbers in each procedure correspond to the numbered marks on the figure beside it.

Screenshots and marks are produced automatically from the running application, so they correspond to the interface as built, not to a redrawing of it. Chapter 16 describes how they are regenerated.

2 What the application does

The workbench reproduces the complete path between an electrical network and an operational decision: the field measurement, the RTU that maps it to a telemetry point, the communication channel that delays or drops it, the SCADA real-time database that stamps it with quality and age, the topology processor that decides whether a breaker is open or closed, the state estimator that solves the weighted least squares problem on the resulting measurement vector, and the optimal power flow computed on the estimated state.

The purpose is not only to obtain a state estimate, but to make visible what each layer did to the data before the result was produced. Every stage can therefore be perturbed, with measurement noise, gross errors, line parameter errors, falsified breaker status or a compromised RTU, and the consequence observed downstream.

The application runs entirely on the local machine. The backend is a Python process (FastAPI, pandapower, OpenDSS) and the interface is served locally by it. No measurement or network data leaves the computer; the only network access is the check for application updates.

2.1 Tabs and their roles

Tab	Role
Topology	Defines the network: test system, buses, lines, meters and switch status. All other tabs operate on it.
Power Flow	Solves the network without measurements. Provides the reference state.
State Est.	Runs the estimator alone, on measurements generated from the reference state.
Bad Data	Detection, identification and correction of gross errors and line parameter errors.
Workbench	Runs power flow, estimation and bad data detection in sequence and compares them.
Pipeline	The full measurement chain with every layer exposed, and fault injection per layer.
Console	The same chain presented as a control room display.
Agents	Distributed estimation over network partitions, with cross-layer agents.
Help	Reference documentation: algorithms, equations, sensor classes, glossary, case studies.

3 Installation and startup

3.1 Installed application (Windows)

1. Download the installer from the most recent release and run it. Python and Node.js are not required: the backend is bundled inside the application.
2. Start **DSSE Workbench** from the Start menu. The application starts its own backend on a local port and loads the interface.
3. When a new version is available, an update banner appears at the top of the window. Accepting it downloads and installs the update.

3.2 Development environment

On a machine with the repository cloned, the virtual environment created and the frontend dependencies installed:

```
.\scripts\run_app.ps1        (or scripts\run_app.bat)
```

The script starts the backend on port 8000 and the frontend development server on port 5173, and opens a browser on the interface. Closing both terminals stops the application.

> **NOTE**
> Changes to the application source do not reach installed copies on their own. An installed application only sees a change after a release is published, which requires the version number in the Electron package manifest to be increased and a matching tag pushed. Committing alone publishes nothing.

4 The application window

The header is the only part of the window that does not change between tabs.



Figure 4.1 — Application header.

1. **Tab bar.** Nine tabs in a fixed order. The active tab is highlighted. Switching tabs does not discard results already computed in the previous one.
2. **Search docs** (`Ctrl` + `K`). Searches the built-in reference from any tab. Selecting a result opens the corresponding topic in the Help tab.
3. **Active network.** Shows the network currently loaded. It applies to every tab, is selected in the Topology tab, and is retained between sessions.

5 Defining the network

Tab: **Topology**. Everything defined here (network, bus and line data, meters, switch status) is used by every other tab, and is retained when the application is closed.

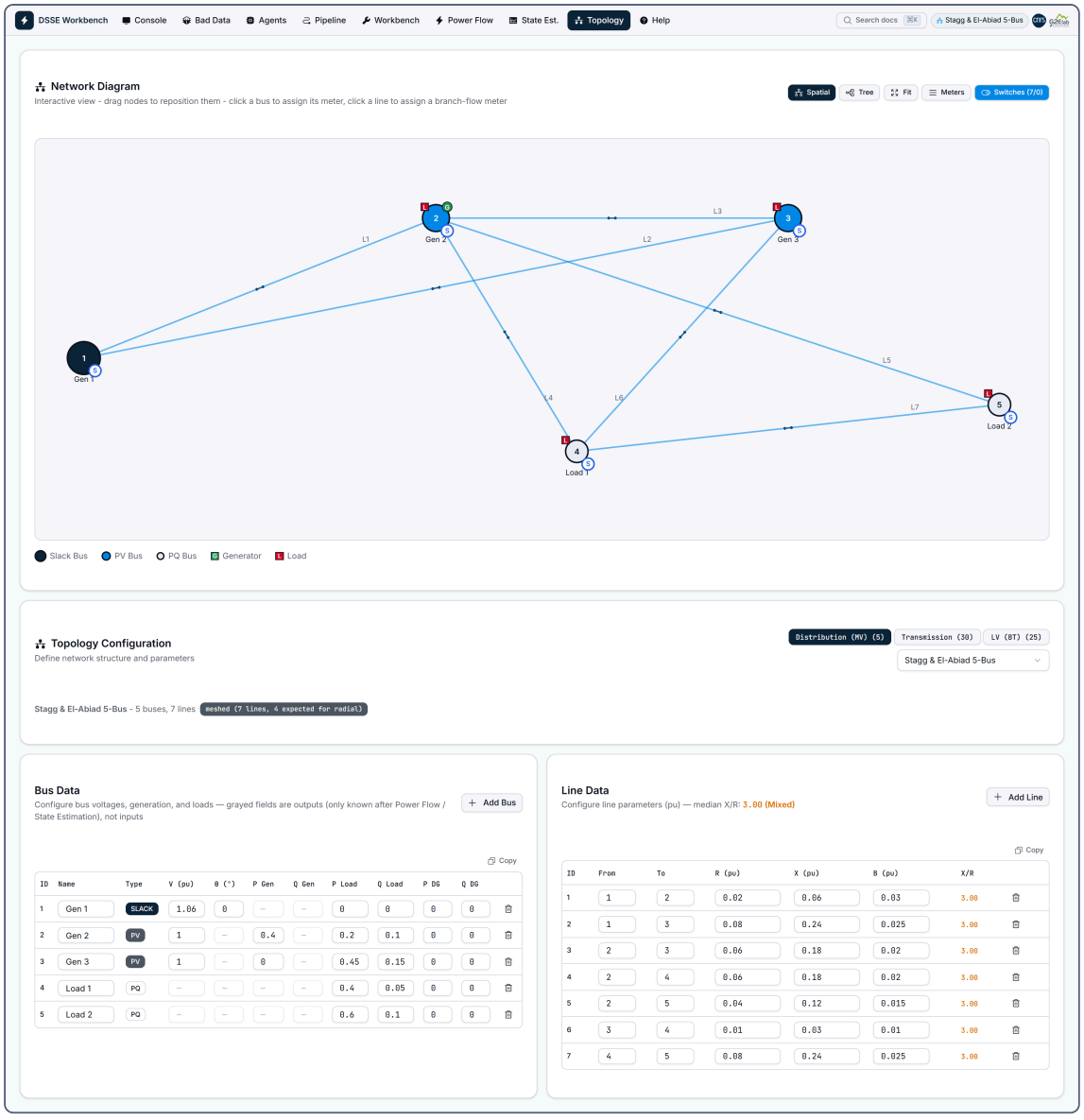


Figure 5.0 — The **Topology** tab in full. Network selection, diagram and the data tables, in one screen. The panels described in this chapter are all located in this screen.

5.1 Selecting a test system

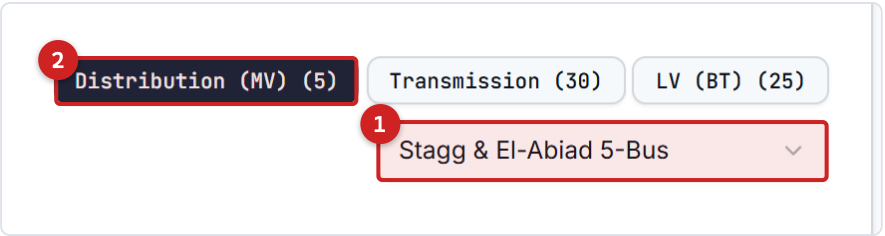


Figure 5.1 — Network selection.

1. **Built-in systems.** Includes the Stagg and El-Abiad 5-bus system (the default), the IEEE 14-bus system and the IEEE 39-bus New England system. The 5-bus system is small enough for results to be verified by hand and is the recommended starting point.
2. **pandapower cases**, grouped by voltage class: distribution (medium voltage), transmission and low voltage. These are built by the backend when selected, so large cases take a few seconds to load.

5.2 The network diagram



Figure 5.2 — Network diagram and its controls.

1. **Spatial** view: buses at their coordinates, as in a single-line diagram. Nodes can be dragged to improve readability.
2. **Tree** view: the same network drawn as a radial feeder, from the source downwards. Useful on distribution networks.
3. **Fit** re-frames the diagram without discarding the current layout.
4. **Meters** opens the measurement assignment panel (section 5.3).
5. **Switches** opens the switch panel and reports the number of closed and open switches (section 5.4).

The diagram follows the usual convention for feeder figures: buses are nodes, branches are edges, voltage is mapped to node colour and loading to edge width. Clicking a bus assigns a meter to that bus; clicking a line assigns a branch-flow meter.

5.3 Assigning meters



Figure 5.3 — Measurement assignment panel.

1. **Apply to all.** Assigns one measurement class to a whole group at once, rather than bus by bus.
2. **Measurement class and target.** The class determines the accuracy used by the estimator; the target restricts the assignment to buses, to lines, or to both.
3. **Individual assignment.** Each bus and each line can be given, or denied, a meter individually. A dash indicates an unmeasured element.

> **WHY THIS MATTERS**
>
> The measurement set determines whether the network is observable and which measurements are critical. Removing a single meter can make a measurement critical, meaning its error becomes undetectable. The *Critical Measurements & Observability* topic in the Help tab describes the criterion.

5.4 Switch status

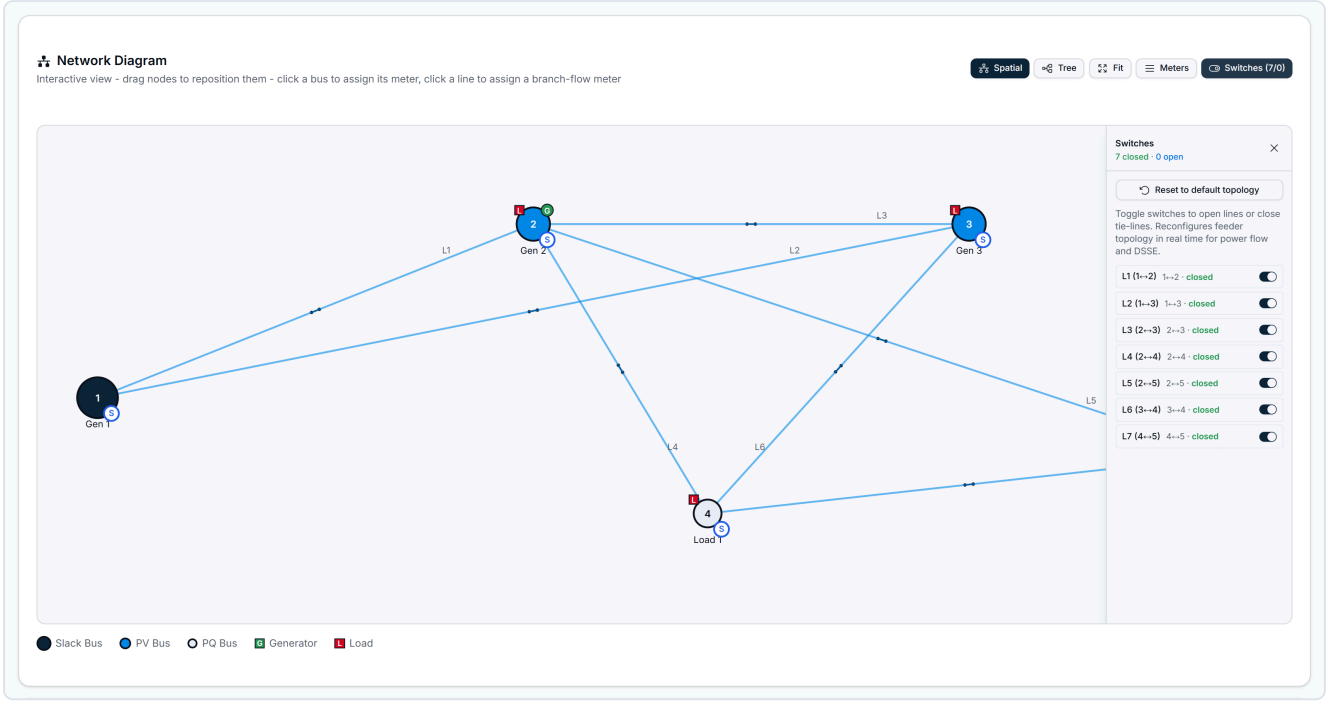


Figure 5.4 — Switch panel; switches can also be operated directly on the diagram.

Switches can be opened and closed from this panel or from the symbols on the diagram. Reconfiguring the network changes which buses are energised and therefore changes every subsequent result.

In the Pipeline and Console tabs, switch status is not a boolean but a double-bit value, following the DNP3 and IEC 61850 convention: 0 intermediate, 1 open, 2 closed, 3 indeterminate. Codes 1 and 2 are conclusive readings; 0 and 3 are not. The distinction that matters operationally is conclusive against inconclusive, not open against closed.

5.5 Bus and line data

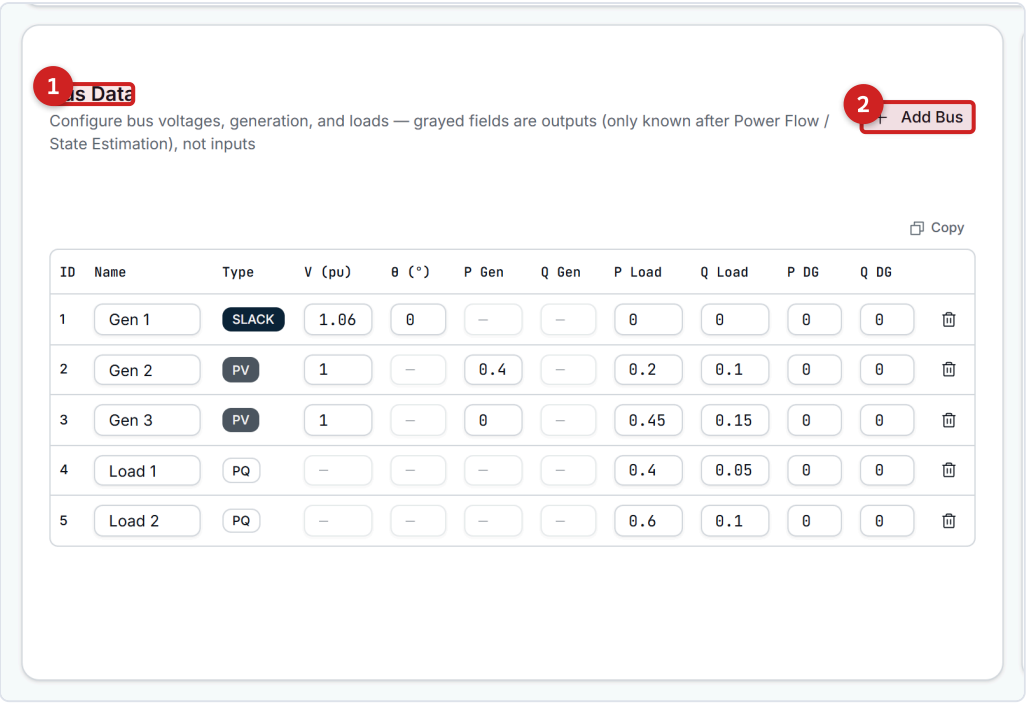


Figure 5.5 — Bus data.

1. **Bus table.** Voltage, angle, generation and load per bus. Greyed fields are outputs, not inputs: they are only known after a power flow or an estimation and cannot be edited. A PQ bus never takes a voltage setpoint, a PV bus never takes an angle, and reactive generation is always the quantity left free.
2. **Add Bus** appends a bus to the network. New buses must be connected by a line before the network can be solved.

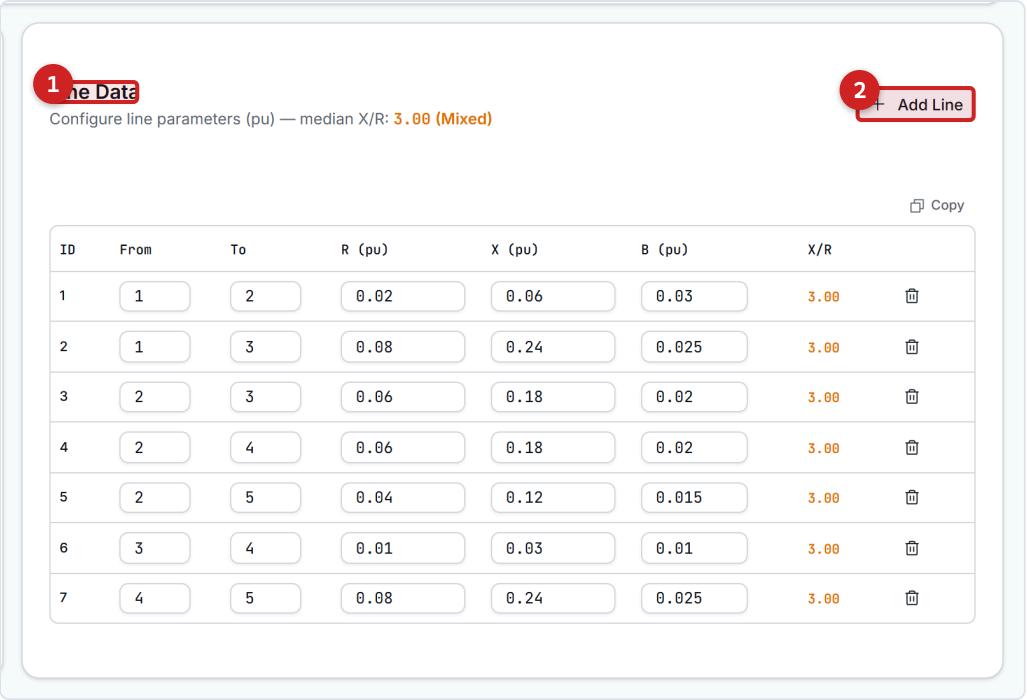


Figure 5.6 — Line data.

1. **Line table.** Terminals and series impedance of each branch. The R/X ratio is reported, because it determines whether the DC approximation is acceptable on this network.
2. **Add Line** appends a branch between two existing buses.

> **CAUTION**
> The edited network is stored and reloaded the next time the application starts. To return to a known state, reload one of the built-in systems from the selector in section 5.1.

6 Running a power flow

Tab: **Power Flow**. Solves the network with no measurements and no estimator. The result is the reference state against which estimation results are compared.

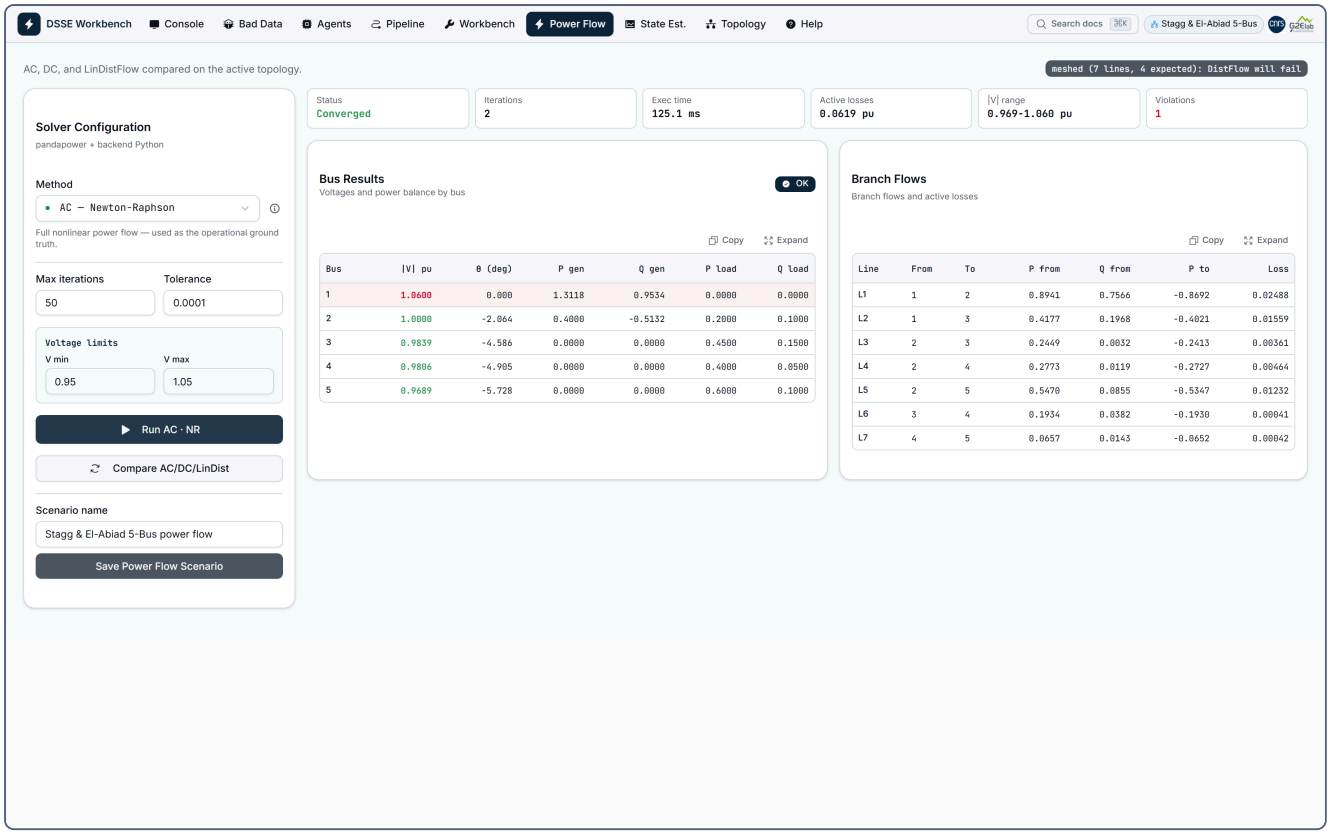


Figure 6.0 — The **Power Flow** tab in full. Solver configuration above, results below. The panels described in this chapter are all located in this screen.

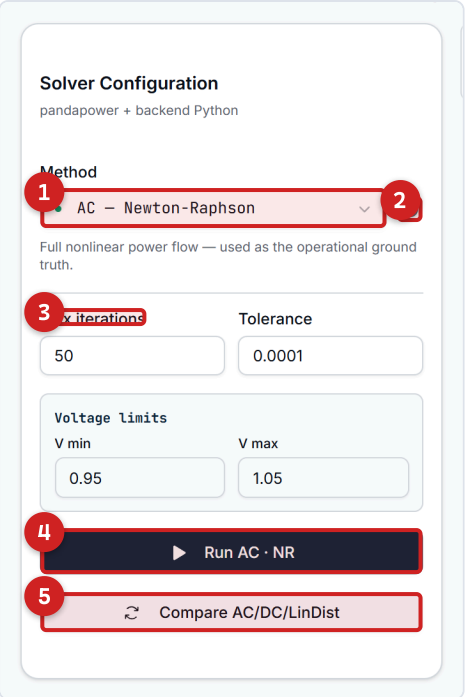


Figure 6.1 — Solver configuration.

1. **Method.** AC Newton-Raphson, linearised DC, or LinDistFlow.
2. **Method description.** States the assumptions of the selected method and the conditions under which they cease to hold.
3. **Convergence settings.** Maximum iterations and tolerance, plus the voltage limits used to flag violations in the results.
4. **Run.** Solves the network and fills the result tables.
5. **Compare AC/DC/LinDist.** Runs all three methods and reports the error of each against the AC solution.

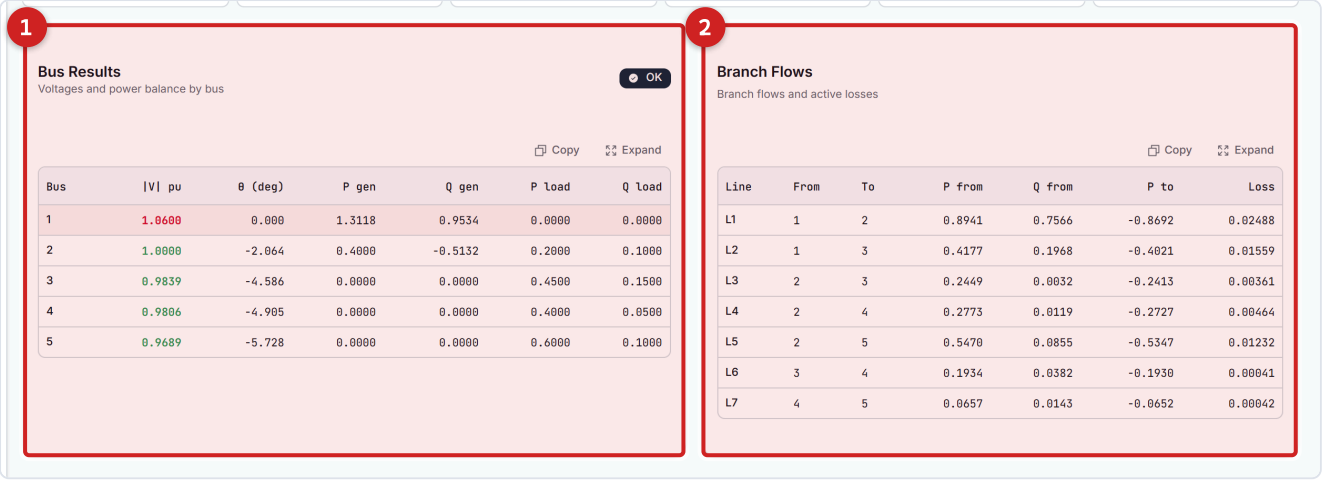


Figure 6.2 — Results after a successful run.

1. **Bus Results.** Voltage magnitude, angle, generation and load per bus, with the convergence status shown beside the table.

2. **Branch Flows.** Active and reactive flow and loading per branch. Every table in the application has a copy control that places its contents on the clipboard in a form that can be pasted into a spreadsheet.

> **INTERPRETATION**
>
> LinDistFlow assumes a radial network and may fail to converge on a meshed one. That outcome is expected behaviour, not a defect, and is one of the reasons the comparison exists.

7 Estimating the state

Tab: **State Est.** Runs the estimator in isolation: measurements are generated from the reference state, noise is added, and the state is reconstructed from them.

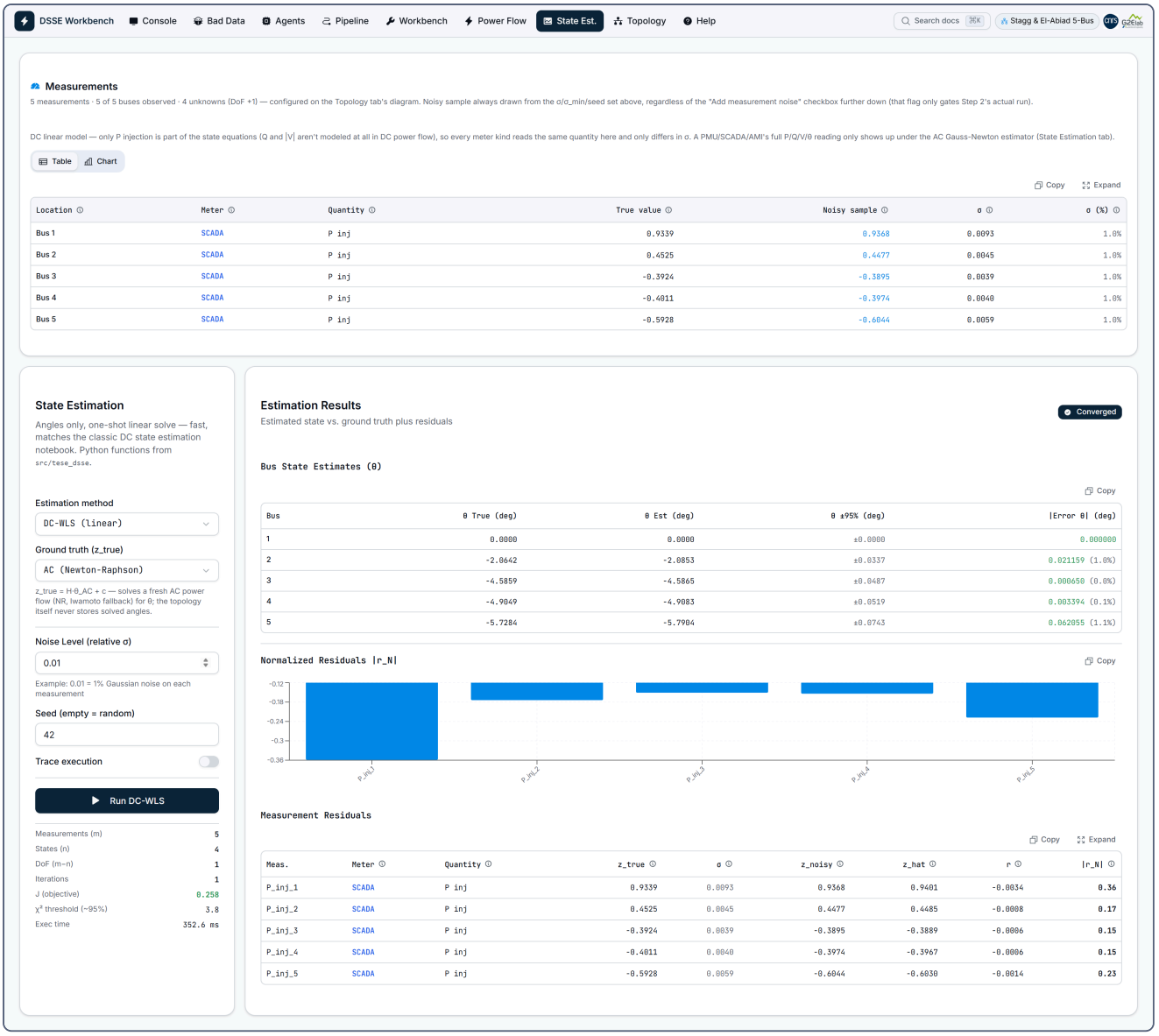


Figure 7.0 — The **State Est.** tab in full. Configuration, then the estimated state and its residuals. The panels described in this chapter are all located in this screen.

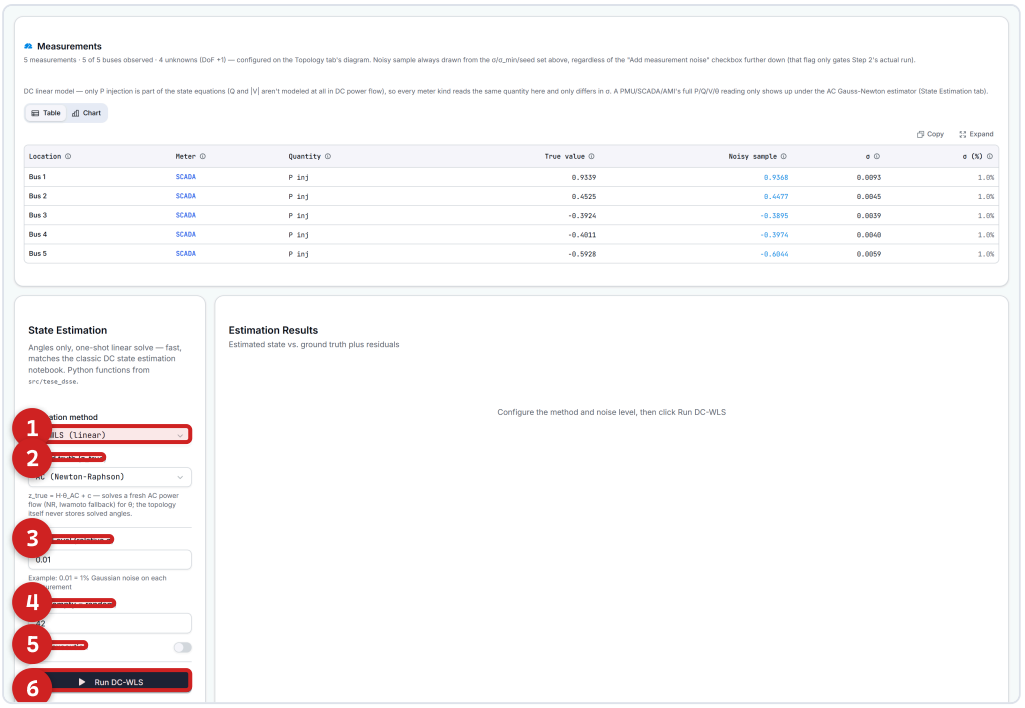


Figure 7.1 — Estimator configuration.

1. **Estimation method.** DC-WLS solves a single linear system for bus angles. AC uses Gauss-Newton iterations over voltage magnitudes and angles.
2. **Ground truth.** Selects the reference state from which the measurement vector is generated. This is what makes the comparison against a known state possible.
3. **Noise level.** The relative standard deviation applied to the measurements.
4. **Seed.** With a fixed seed the run is exactly reproducible; left empty, each run draws a new noise realisation.
5. **Trace execution.** Enables the step-by-step execution panel, which reports each iteration and the intermediate matrices. It is slower and is intended for study and verification.
6. **Run.** Solves and fills the result panel.

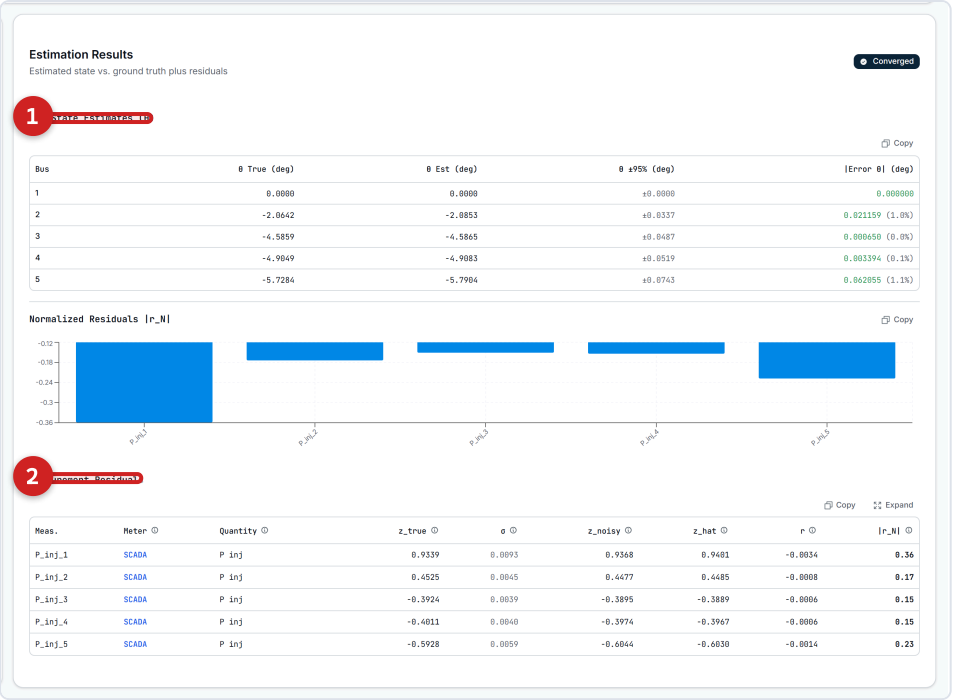


Figure 7.2 — Estimation results.

1. **Bus State Estimates.** The estimated state beside the ground truth, with the deviation per bus.
2. **Measurement Residuals.** Normalised residuals per measurement, together with the objective function history, which should decrease and then flatten. A residual history that does not settle indicates that the measurement set, and not the solver, is the problem.

> **RECOMMENDED PRACTICE**
>
> Fix the seed whenever two configurations are being compared. Without it, part of the difference observed between two runs is only the noise realisation.

8 Bad data analysis

Tab: **Bad Data**. Detection, identification and correction of gross measurement errors and line parameter errors, in the geometric formulation of Bretas. The procedure has four stages and they are performed in order.

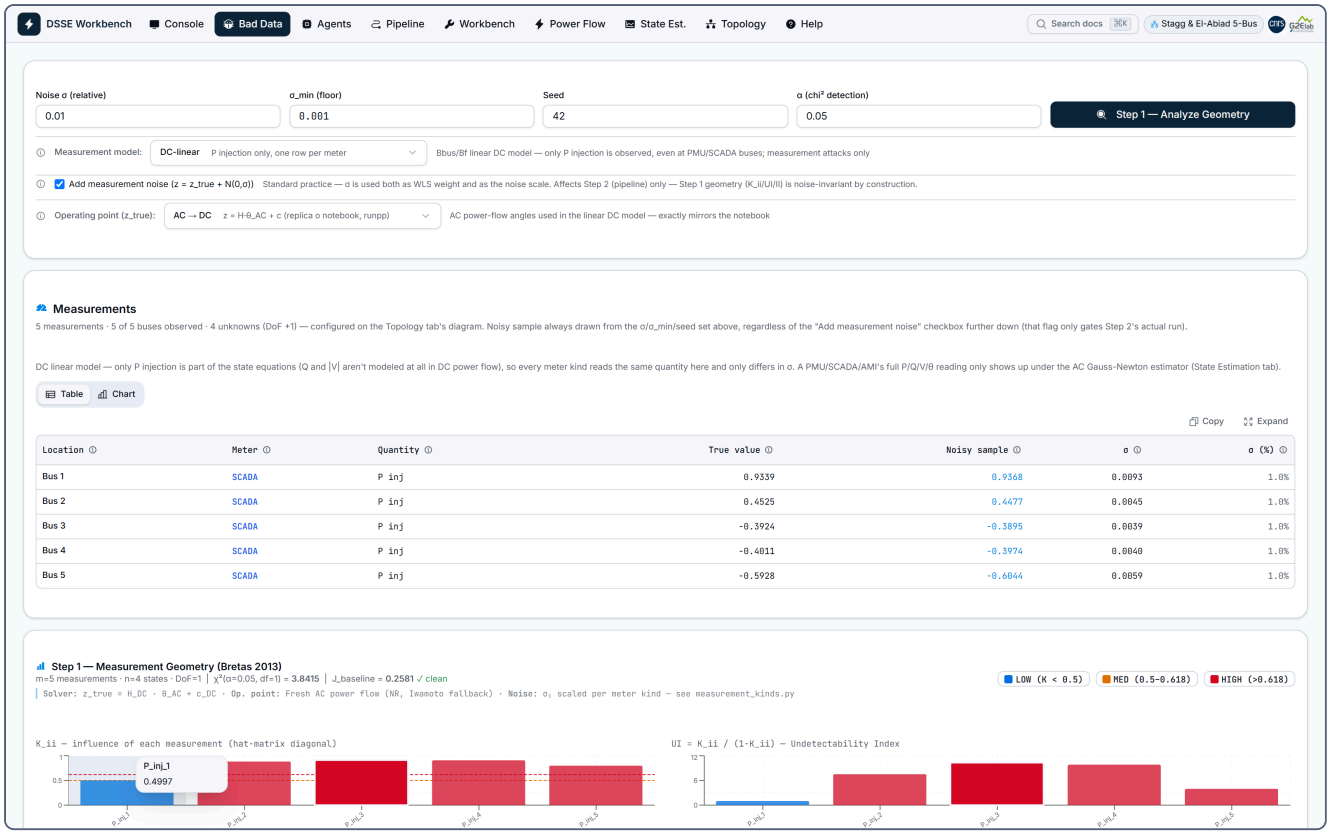


Figure 8.0 — The **Bad Data** tab as it opens. The four stages run down the page in order; the full page is several screens tall, so this shows the top of it. The panels described in this chapter follow below it.

8.1 Stage 1: measurement geometry



Figure 8.1 — Experiment settings.

1. **Noise σ** and the σ floor, which set the measurement accuracy assumed by the estimator.
2. **α.** Significance level of the chi-square detection test.
3. **Measurement model.** DC-linear, with one row per meter, or AC → DC, which reproduces the reference notebook. Parameter attacks require the AC model.
4. **Analyze Geometry.** Computes the geometry of the measurement set before any error is injected.

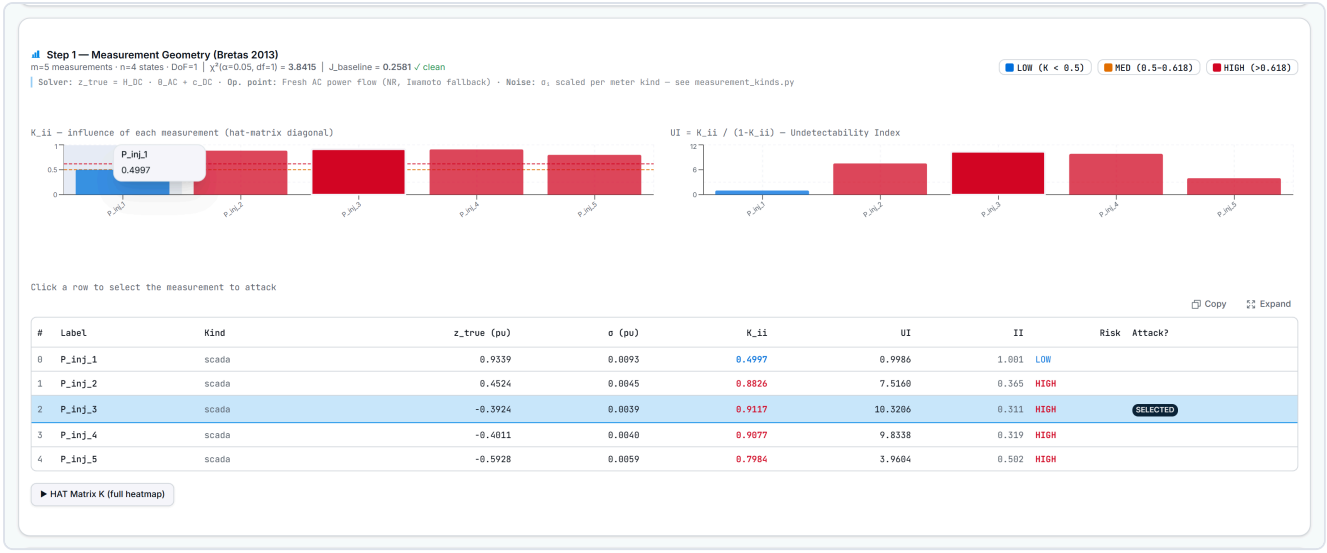


Figure 8.2 — Measurement geometry, before any injection.

The table reports, per measurement, the diagonal of the hat matrix (K_{ii}), the undetectability index, the innovation index and the degrees of freedom of the test. It identifies, in advance, which measurements are able to hide an error. Select a row to use that measurement as the attack target in the next stage.

8.2 Stage 2: configuring the error

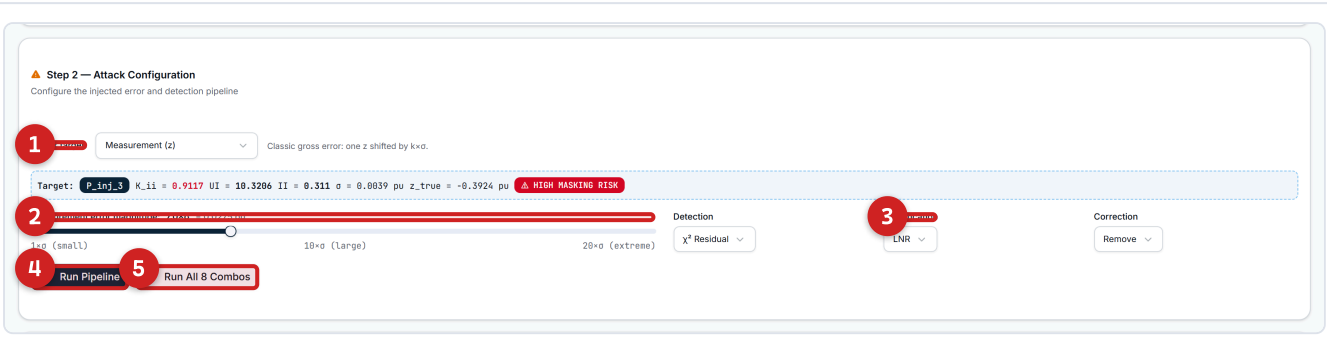


Figure 8.3 — Attack configuration.

1. **Attack target.** A measurement, a line parameter, or both. The last two require the AC model. The summary beside the selector reports K_{ii}, the undetectability and innovation indices, σ and the true value of the selected measurement, and warns when the masking risk is high.
2. **Error magnitude**, in multiples of σ for a measurement error, or as a percentage of the parameter for a line error.
3. **Pipeline.** Three independent choices: detection (chi-square on the residual or on the composed measurement error), identification (largest normalised residual, normalised composed error, or by line in the AC case) and correction (remove the measurement, replace it by its true value, or correct the line parameter). Identification by line combined with correction by parameter is the sequence that repairs the line instead of discarding valid data.
4. **Run Pipeline.** Executes the selected combination.
5. **Run All 8 Combos.** Executes the fixed 2×2×2 grid on the same injection, so that strategies are compared without changing the experiment underneath them.

8.3 Stage 3: reading the result

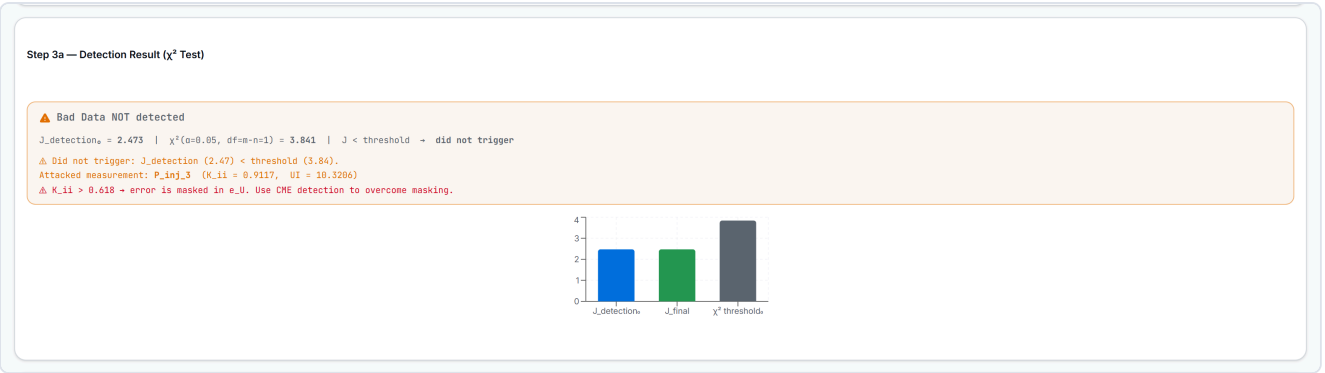


Figure 8.4 — Detection result.

The result panel reports whether the chi-square test triggered, which measurement the identification step selected, whether that measurement is the one actually corrupted, and the state obtained after correction. Below it are the scatter plot of residuals (the corrupted measurement in red, false flags in orange), the hat matrix heat map, the complete per-measurement table and, for parameter errors, the line signature ranking and the corrected parameter.

> **CAUTION WHEN READING RMSE**
>
> The comparison against the true state exists only because this is a validation environment. In operation there is no true state available to decide a correction. A lower RMSE does not mean a more correct repair: a correction that discards the measurement can look numerically better while removing valid data and leaving the line parameters permanently wrong.

9 Running several methods at once

Tab: **Workbench**. Runs power flow, state estimation and bad data detection as a chain of blocks and presents the results side by side.

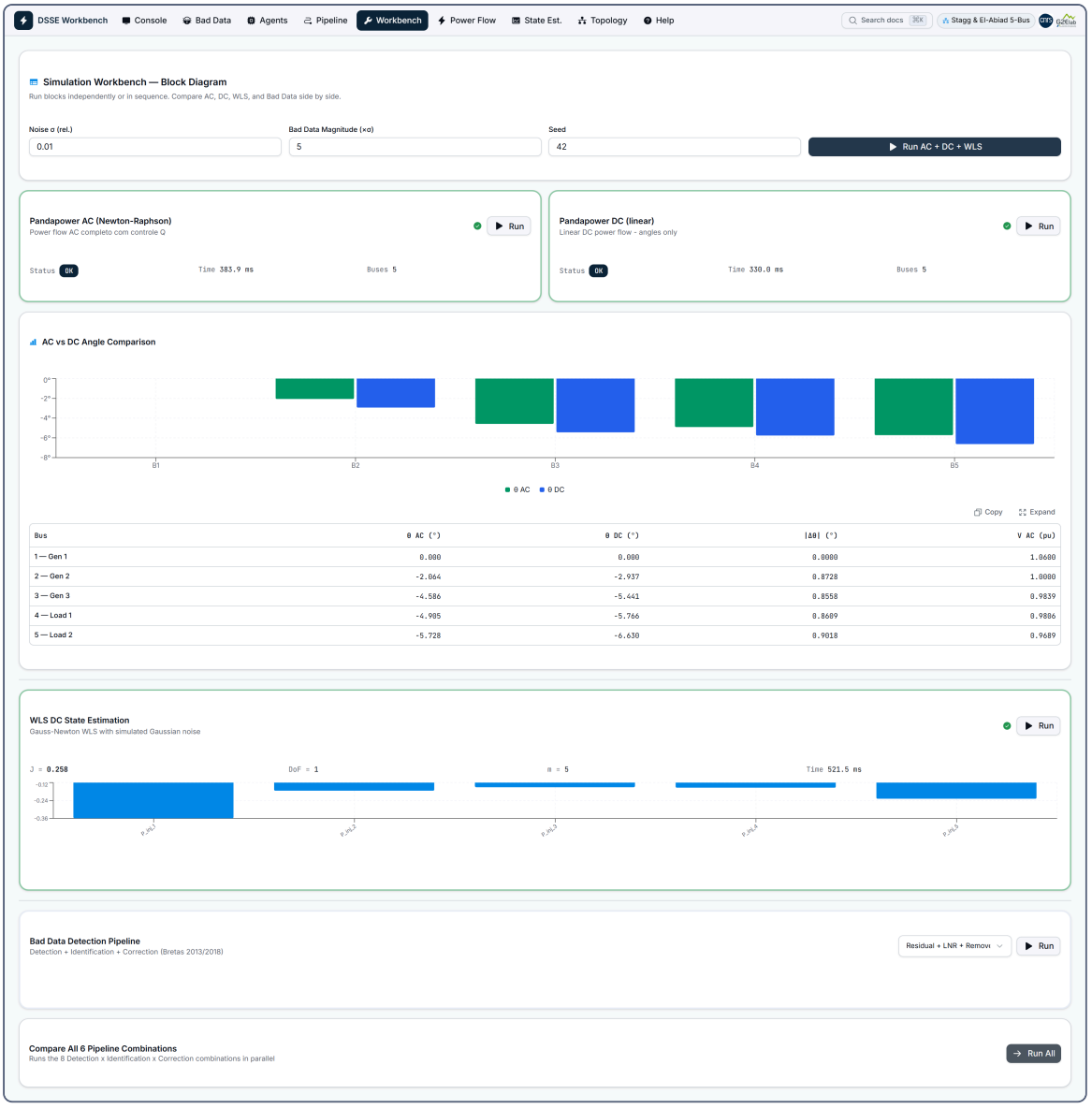


Figure 9.0 — The **Workbench** tab in full. Each block reports its own result, side by side. The panels described in this chapter are all located in this screen.

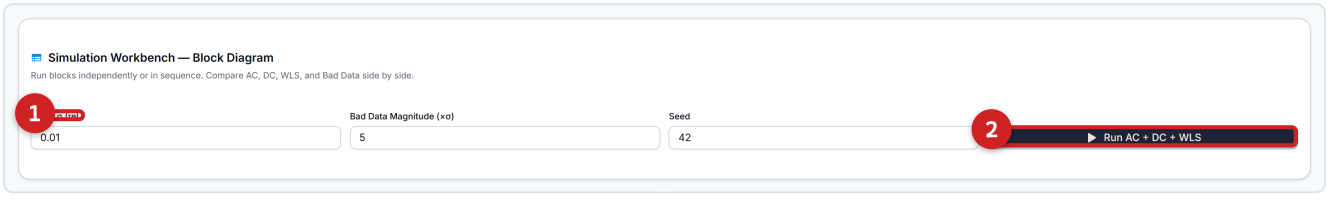


Figure 9.1 — Block diagram and shared settings.

1. **Shared settings.** Noise level, gross error magnitude and seed, applied to every block in the chain.

2 **Run AC + DC + WLS.** Executes the chain in one step. Each block also has its own run control, which is the faster way to repeat a single stage after changing its input.



Bus	θ AC (°)	θ DC (°)	|Δθ| (°)	V AC (pu)
1 — Gen 1	0.000	0.000	0.0000	1.0600
2 — Gen 2	-2.064	-2.937	0.8728	1.0000
3 — Gen 3	-4.586	-5.441	0.8558	0.9839
4 — Load 1	-4.905	-5.766	0.8609	0.9806
5 — Load 2	-5.728	-6.630	0.9018	0.9689

Figure 9.2 — AC and DC angles compared.

The comparison chart shows the bus angles obtained by both methods, which is the direct way to see where the linear approximation stops being valid on the network in use. The blocks below report the normalised residuals of the estimator and the outcome of the detection pipeline. **Run All** sweeps the pipeline combinations instead of a single one.

10 The measurement chain

Tab: **Pipeline**. The complete chain from field to control action, with every layer exposed and its own result table, and with faults injected at the layer where they would actually occur.

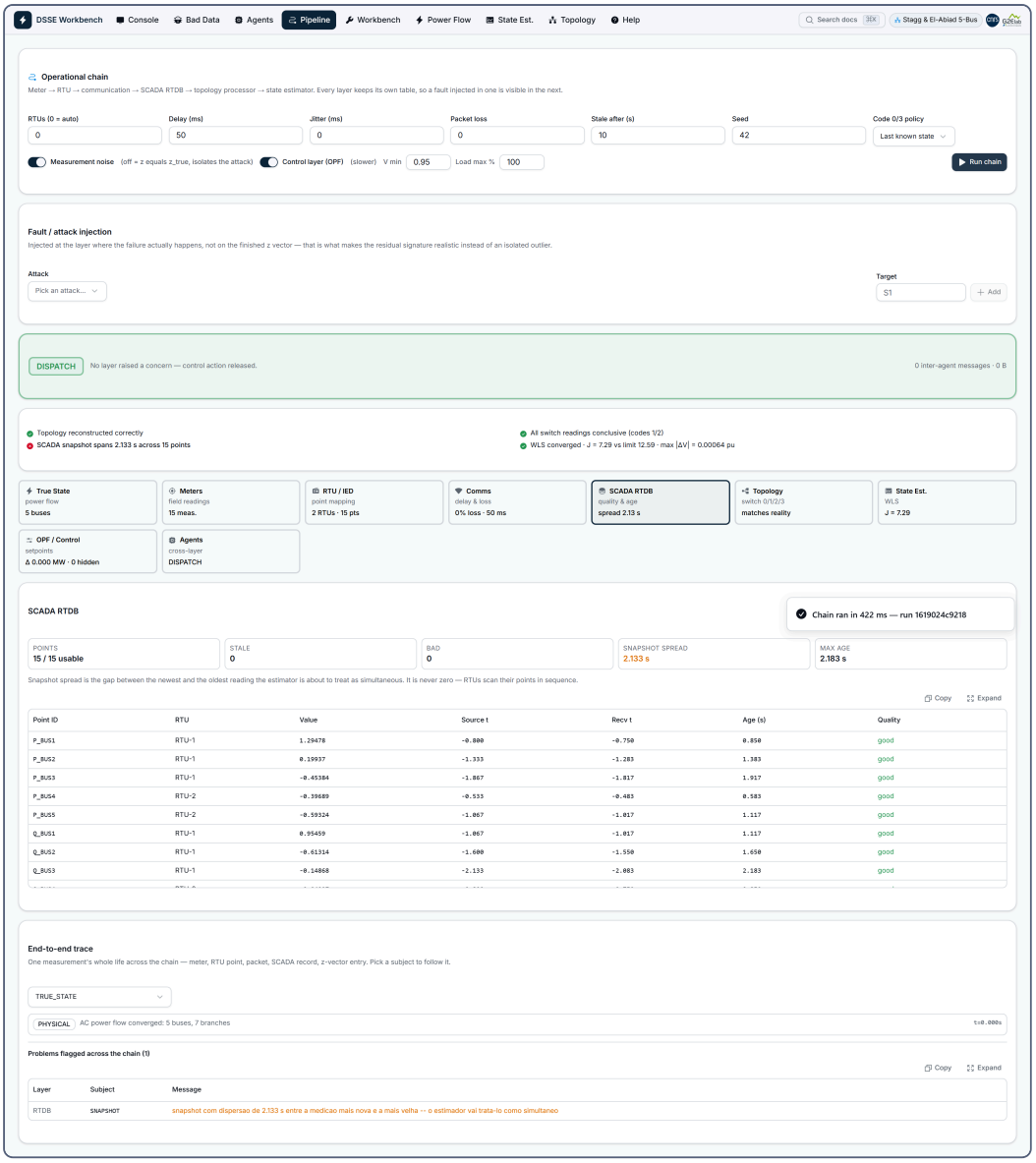


Figure 10.0 — The **Pipeline** tab in full. Configuration, the layer strip, and the table of the selected layer. The panels described in this chapter are all located in this screen.

10.1 Configuring the chain

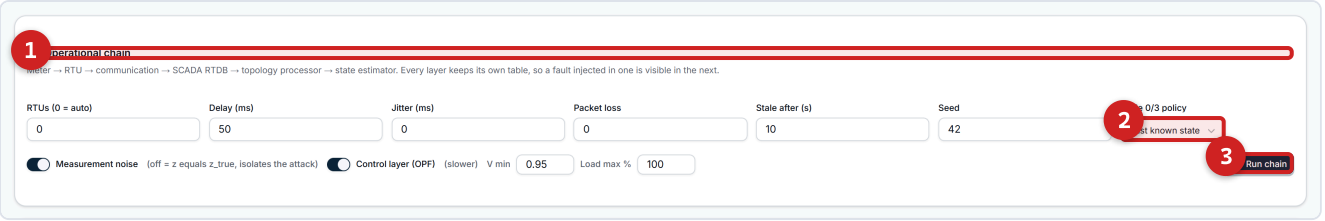


Figure 10.1 — Chain settings.

1. **Layer settings.** Number of RTUs, base delay and jitter of the channel, packet loss rate, the age after which a point is considered stale, and the seed. Measurement noise and the control layer (OPF, with its voltage and loading limits) are enabled separately.
2. **Unknown switch policy.** What the topology processor does with an inconclusive breaker reading: keep the last known state, assume closed, assume open, or infer the state from the measured flow.
3. **Run chain.** Executes the chain end to end.

10.2 Injecting a fault

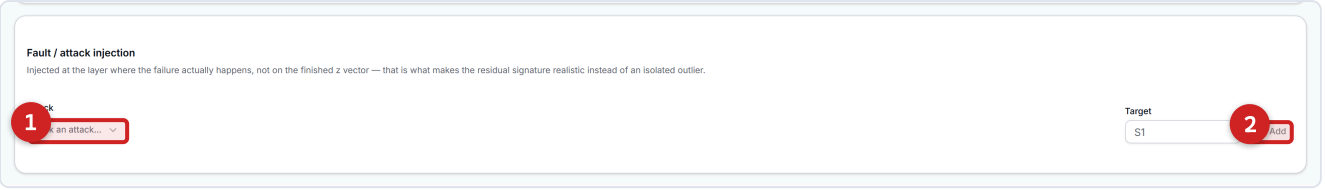


Figure 10.2 — Fault and attack injection.

1. **Fault selection.** Each fault has its own target (an RTU, a switch, a sensor) and its own parameters, which appear once it is selected.
2. **Add.** Appends the configured fault to the list. Several faults can be active in the same run; *Clear all* empties the list.

> **WHY INJECTION PER LAYER**
>
> Faults are injected at the layer where the failure would physically occur, not on the finished measurement vector. This is what makes the resulting residual signature realistic, instead of an isolated outlier that any detector would find.

10.3 Following the data

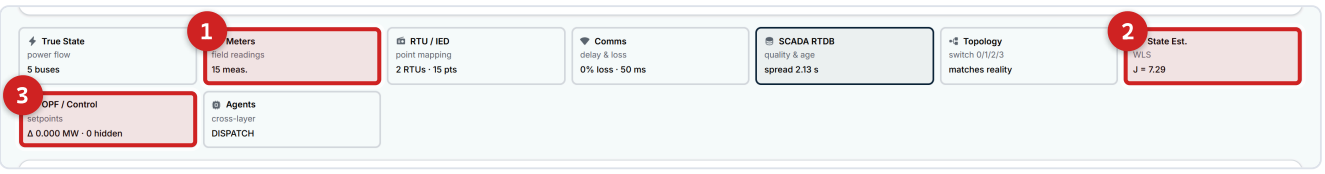


Figure 10.3 — The layer strip after a run.

1. **Field and measurement layers.** The true state and the readings taken from it.
2. **Estimation layer.** Select any block to display that layer's own table below the strip; the strip stays visible, so the chain never leaves the screen.
3. **Control layer.** The setpoints computed on the estimated state, and the violations that remain hidden when the estimate is wrong.

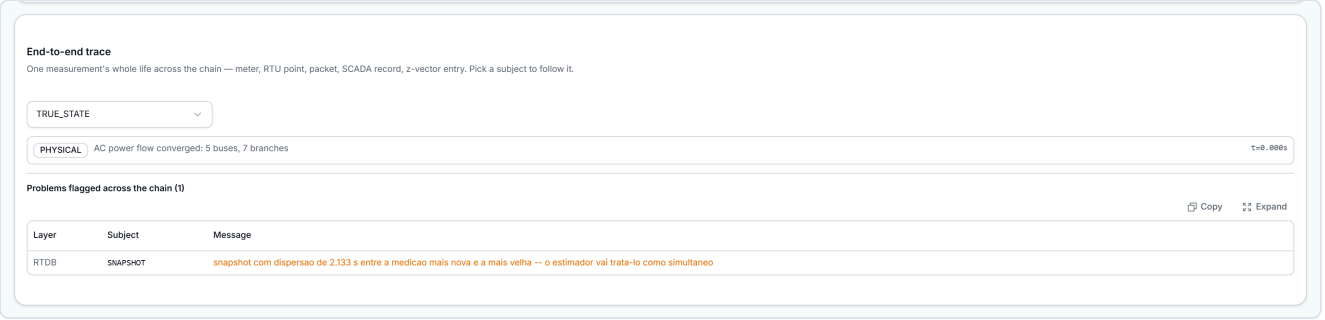


Figure 10.4 — End-to-end trace of one measurement.

Selecting a measurement in the trace panel shows its complete life across the chain: the meter reading, the RTU point, the transmitted packet, the SCADA record and the entry in the measurement vector. Without it, a fault injected in the communication channel is visible only as a large residual.

11 The operator view

Tab: **Console**. The same chain as Chapter 10, presented the way a control room would show it. It runs once automatically when the tab is opened.

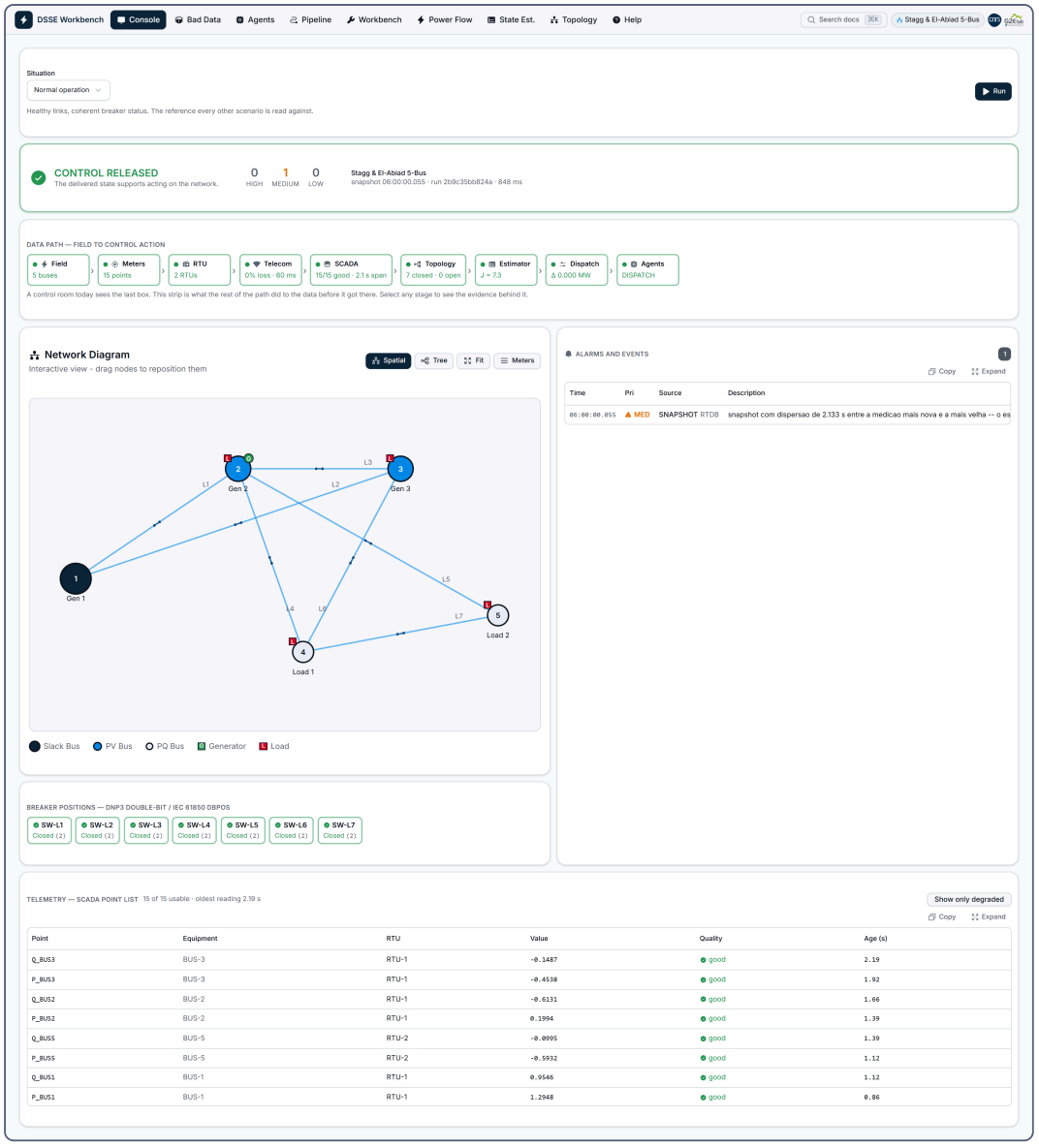


Figure 11.0 — The **Console** tab in full. Status band, chain, diagram, alarms and telemetry. The panels described in this chapter are all located in this screen.



Figure 11.1 — Scenario selection.

1. **Situation.** Six predefined scenarios: normal operation, a degraded link on one RTU, packet loss on the network, a falsified breaker status, disagreeing breaker contacts, and a compromised RTU. Each carries its own settings and faults, so a situation can be demonstrated without configuring anything.
2. **Run.** Executes the chain for the selected scenario.

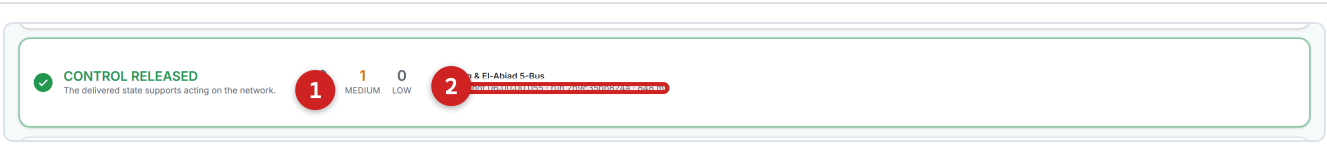


Figure 11.2 — Status band.

1. **Recommendation and alarm counts.** Whether the delivered state supports acting on the network, with the number of alarms by priority.
2. **Run identification.** Network name, snapshot time, run identifier and elapsed time. The snapshot time is the span of the measurement set, not a single instant.

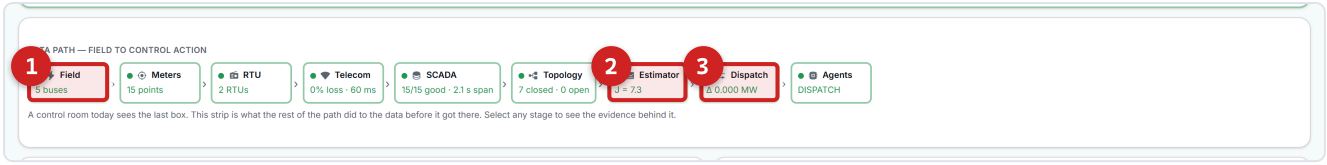


Figure 11.3 — Process mimic of the chain.

1. **Field and acquisition stages.** Each stage carries its own severity colour and a summary figure.
2. **Estimator stage.** Selecting any stage displays the evidence behind its colour immediately below the strip.
3. **Dispatch stage.** A control room today sees only this last box. The strip shows what the rest of the path did to the data before it arrived there.

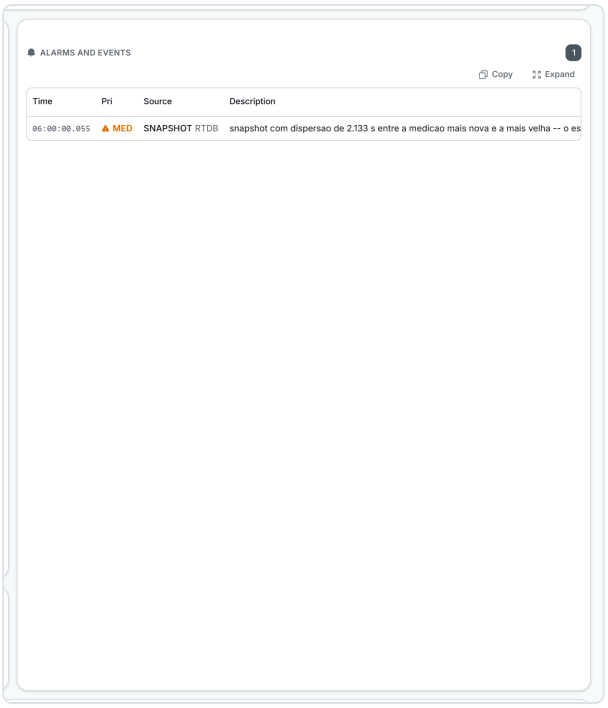


Figure 11.4 — Alarm and event list.

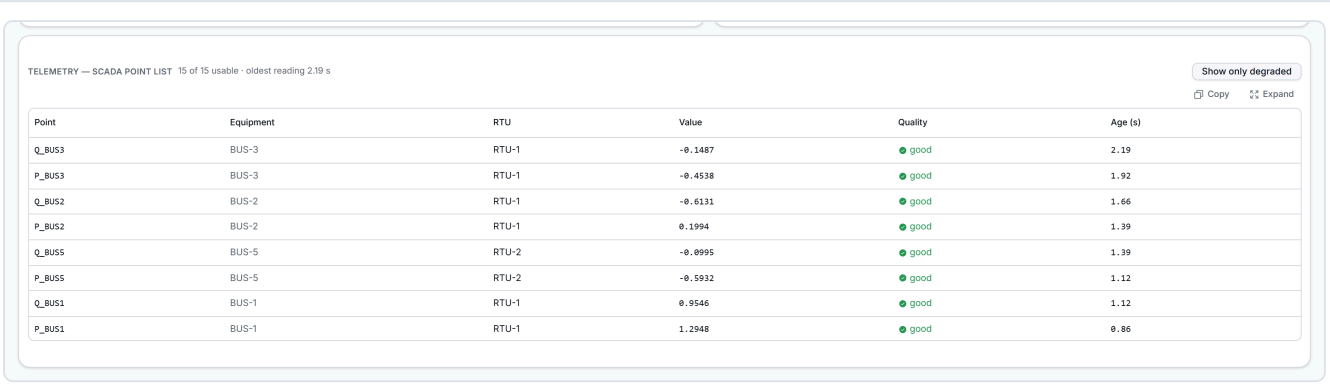


Figure 11.5 — Telemetry point list.

The alarm list is chronological, priority coded and names the source equipment. The telemetry list reports point identifier, equipment, quality flag and data age, which is where a stale value entering the estimator as if it were current becomes visible.

12 Multi-agent estimation

Tab: **Agents**. The network is partitioned into islands, each island estimates its own portion, and agents exchange information across layers.

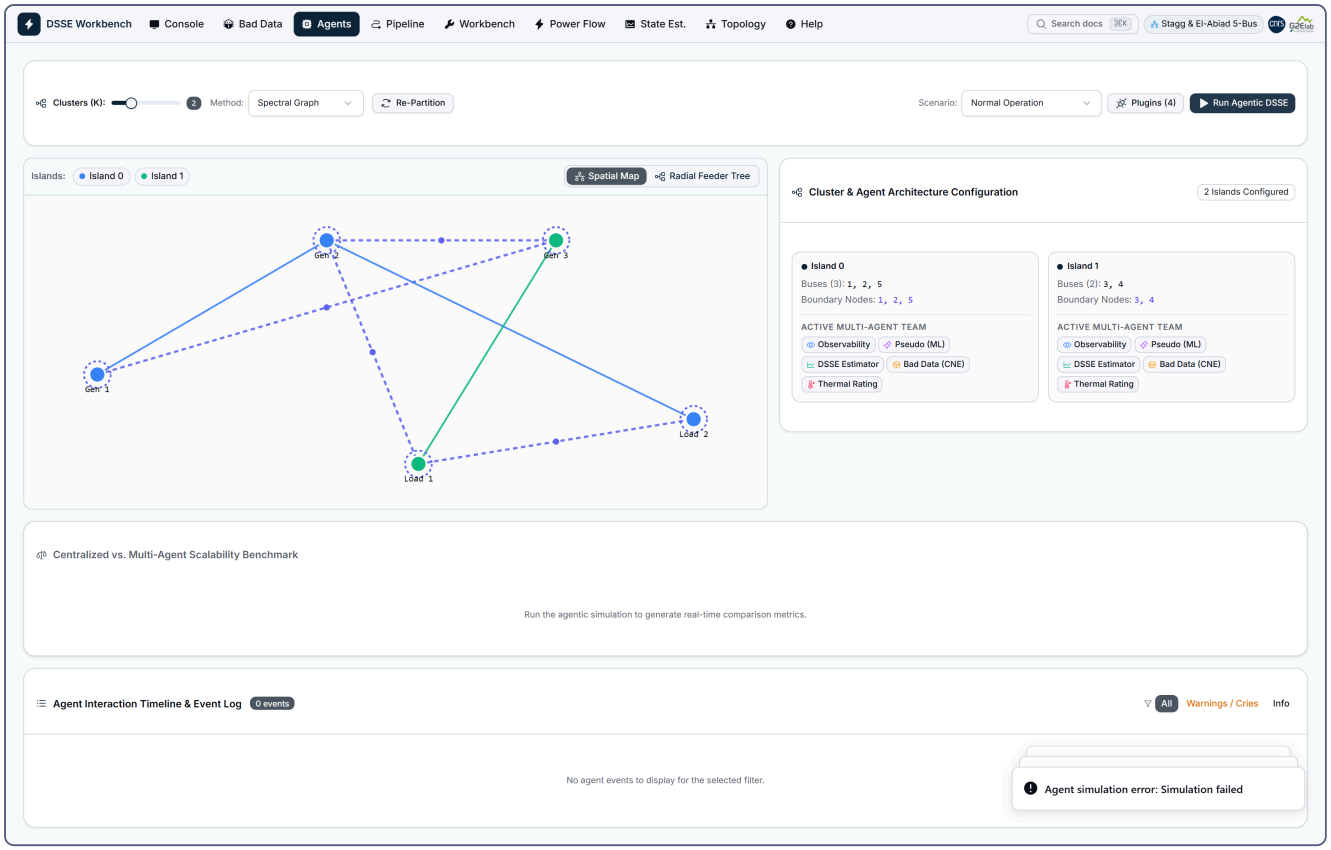


Figure 12.0 — The **Agents** tab in full. Islands on the left, agent activity on the right. The panels described in this chapter are all located in this screen.

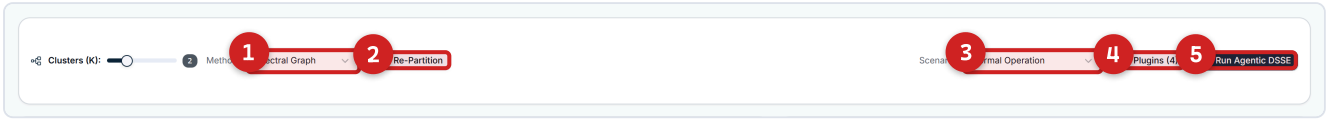


Figure 12.1 — Partitioning and simulation controls.

1. **Partition method.** Spectral graph partitioning, or partitioning along radial feeders. The number of islands is set by the adjacent control.
2. **Re-Partition.** Recomputes the partition. The partition is also recomputed automatically when the tab is opened or the network changes.
3. **Scenario.** The situation simulated during the agent run, using the same set as the Console tab.
4. **Plugins.** Opens the plugin manager, which is the extension point for adding agents without modifying the core.
5. **Run Agentic DSSE.** Runs distributed estimation over the current islands. It is enabled only once a partition exists.

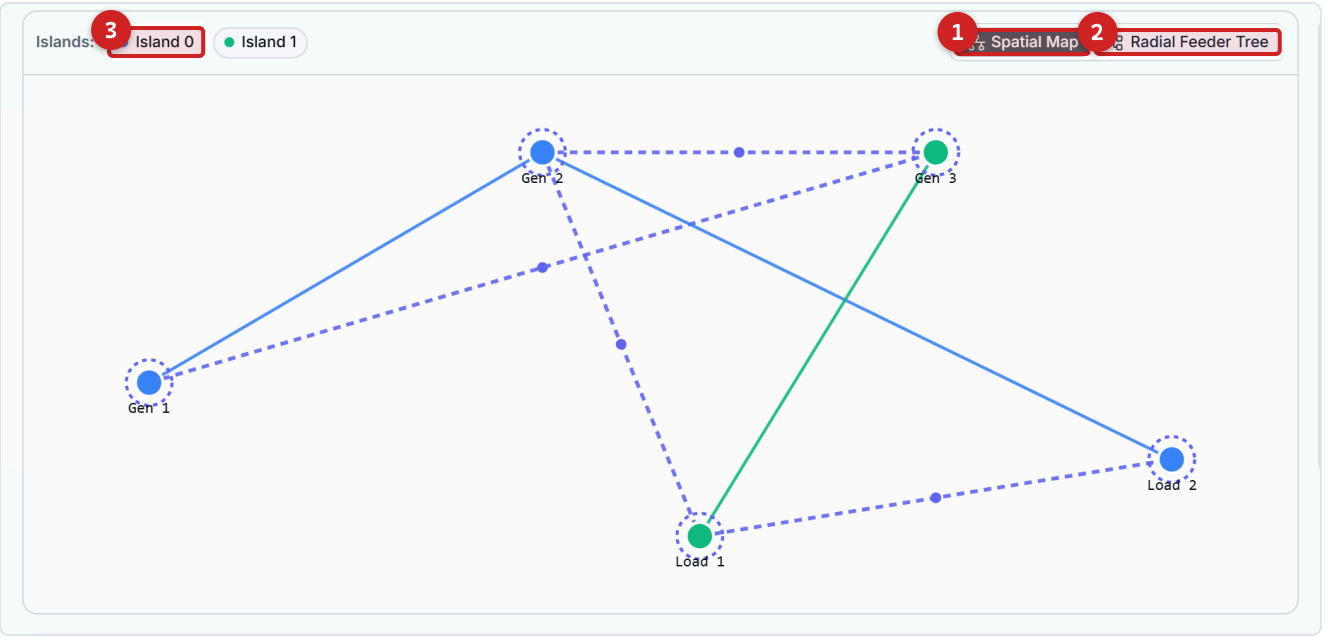


Figure 12.2 — Partitioned network.

1. **Spatial map** of the partitioned network, islands in distinct colours, with the boundary nodes marked.
2. **Radial feeder tree**, the same partition drawn along the feeders.
3. **Island selection.** Isolates one island at a time, to inspect what a single agent sees.

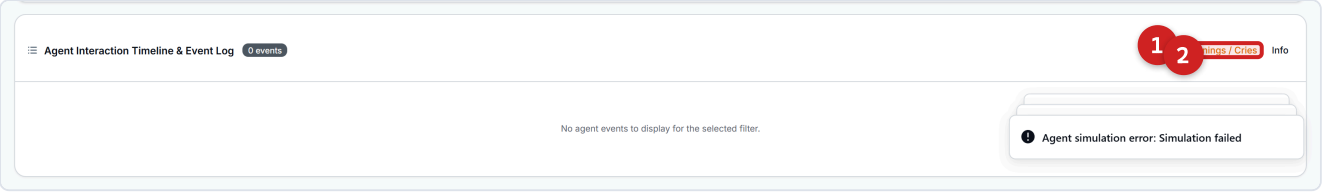


Figure 12.3 — Agent event log.

1. **All events**, in chronological order, with the agent that emitted each one.
2. **Warnings and cries.** The on-demand protocol by which an agent reports that it cannot produce a trustworthy result, instead of returning a silently wrong number. The benchmark panel beside the log compares the distributed run against the centralised one.

13 Built-in reference

Tab: **Help**. The mathematical documentation of the methods implemented in the application.

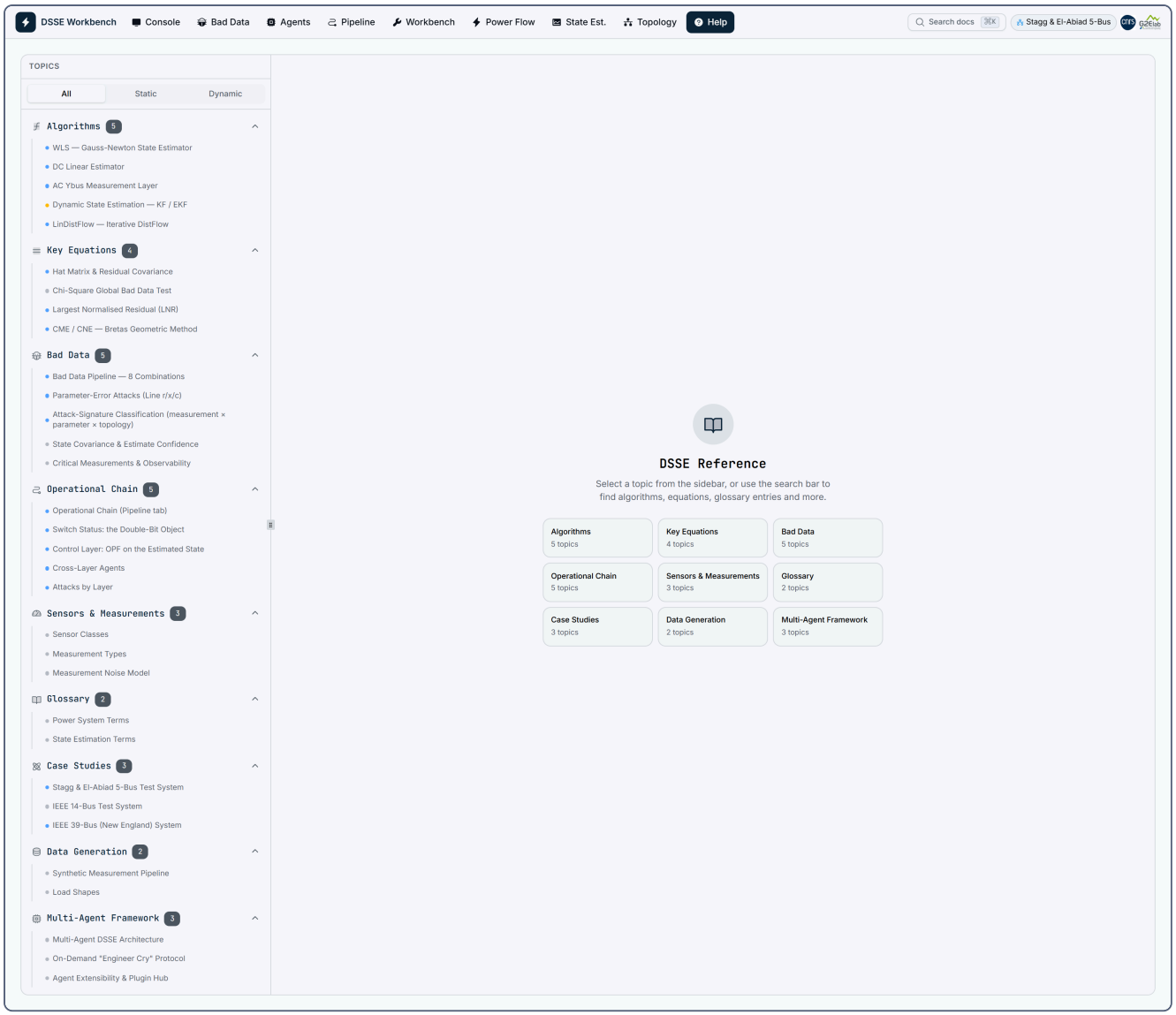


Figure 13.0 — The **Help** tab in full. Categories on the left, the selected topic on the right. The panels described in this chapter are all located in this screen.

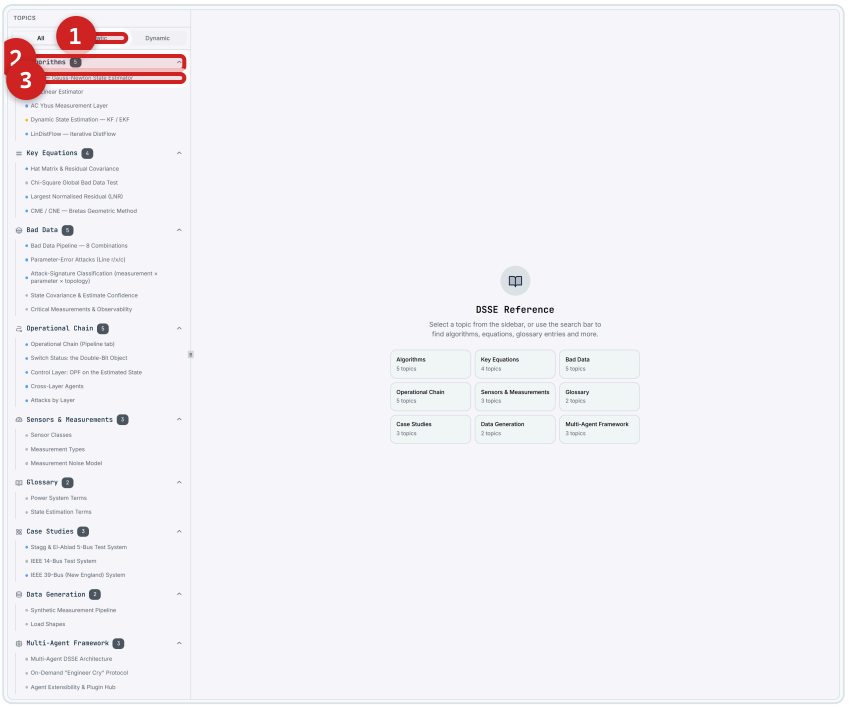


Figure 13.1 — Help navigation.

1. **Paradigm filter.** All topics, static estimation only, or dynamic estimation only (Kalman and extended Kalman filtering).
2. **Categories**, with the number of topics in each: algorithms, key equations, bad data, operational chain, sensors and measurements, glossary, case studies, data generation and the multi-agent framework.
3. **Topic.** Each topic states the formulation with rendered equations, the assumptions and the references. Reaching a topic through the search palette (`Ctrl` + `K`) is usually faster than navigating the list.

14 Conventions and shortcuts

Element	Behaviour
`Ctrl` + `K`	Opens the documentation search from any tab.
Copy control on tables	Places the table contents on the clipboard, formatted for a spreadsheet or a notebook.
Information icon on columns	States what the column contains and how it was computed.
Method description control	States the assumptions and limits of the selected method.
Greyed input fields	Outputs, not inputs: quantities known only after solving.
Seed field	Empty means a new random draw on every run; a fixed value makes the run reproducible.
Diagram conventions	Buses as nodes, branches as edges, voltage as node colour, loading as edge width.
Switch status codes	0 intermediate, 1 open, 2 closed, 3 indeterminate; 1 and 2 are the conclusive readings.

15 Troubleshooting

Symptom	Cause and resolution
Every tab reports a connection error	The backend is not running. In a development environment, check the backend terminal; in the installed application, close and reopen it so that a new local port is selected.
A run control is disabled	A prerequisite step has not been completed. In Bad Data, the pipeline requires the geometry stage and a selected measurement; in Agents, the simulation requires an existing partition.
LinDistFlow does not converge	The method assumes a radial network. Expected on meshed networks.
Loading a large case takes several seconds	Expected: the network is built by the backend, and transmission cases contain far more buses than the 5-bus and 14-bus systems.
Results change between identical runs	The seed field is empty, so each run draws a new noise realisation.
The network is not the expected one after restart	The edited network is retained between sessions. Reload a built-in system from the Topology tab.

16 Maintaining this manual

The figures in this manual, and the position of every numbered mark drawn on them, are generated from the running application rather than written by hand:

```
.venv\Scripts\python.exe scripts\capturar_screenshots_app.py
```

The script opens the application, visits each tab, performs the main action of that tab, and crops the panel documented by each figure. It then queries the document for the rectangle of every marked element and writes those rectangles to `coords.js`, next to the images. This page refers to each mark by name and converts the rectangle to a percentage of the image, so figures and marks are always regenerated together.

Figures are cropped to the panel they document rather than captured as full pages, because this manual is also produced as a PDF and a full-page screenshot is several thousand pixels tall.

16.1 Adding a new mark

1. Add an entry to `MANUAL_FIGURES` in the capture script, or a pin to an existing figure, giving a name and how to locate the element.
2. Reference that name in the `data-pins` attribute of the figure in this document, in the same order as the numbered steps beside it.
3. Run the script. A pin that no longer matches an element is not drawn and is reported in the figure caption, rather than silently pointing at the wrong place.

16.2 Producing a PDF

Open this file in a browser and print to PDF with background graphics enabled, so that the marks and panel backgrounds are preserved. The page layout, the table of contents and the page breaks between chapters are defined for A4.

DSSE Workbench user manual · Université Grenoble Alpes, CNRS, G2Elab · figures generated by `scripts/capturar_screenshots_app.py`.